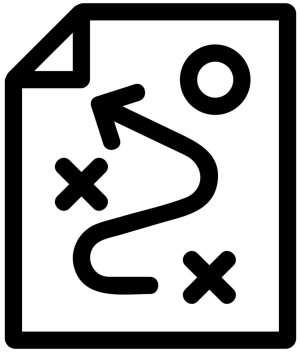
More Synchronization Primitives

CSE 334

Variety of primitives



All Turing equivalent

All provided by the operating system, built using ISLs

All allow for non-busy wait

- While waiting, the core will swap to a different thread
- Also known as a “blocking wait”

Primitives

- Thread join
- Channels
- Mutex
- Semaphore
- Barrier
- RWLock
- CondVar
- Monitor*

Why you should care extra

Most parallel programs will require the proper use of synchronization primitives to build correct code

Homework 3 will involve implementing synchronization primitives!

Thread Join

```
thread_1.join()
```

Forces the current thread to wait until `thread_1` is done.

Channels

Two methods:

- send
- recv

Even a channel that doesn’t send data is useful for synchronization!

Channels

Allows us to establish a causal relationship.

thread 1:

```
x = 1
chan.send()
```

thread 2:

```
chan.recv()
print(x)
```

Mutex

MUTual EXclusion

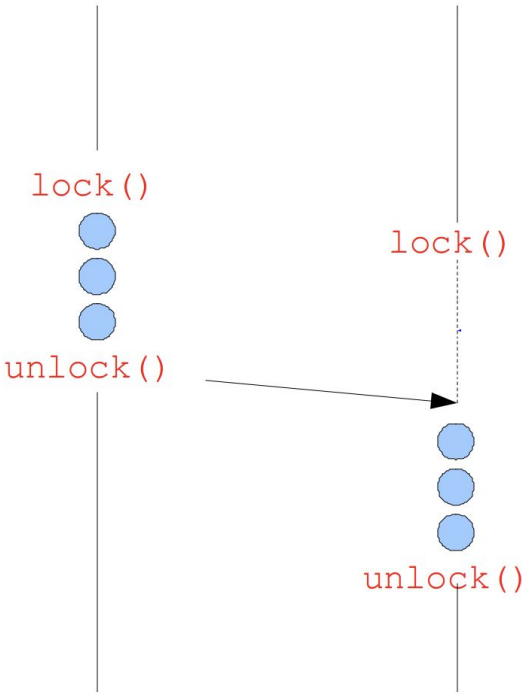
Also known as a SleepLock or a BlockingLock

Mutex

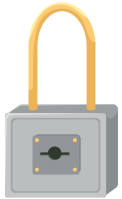
Two methods:

- lock (or acquire)
- unlock (or release)

Only one thread can hold a Mutex at a time!



Mutex



thread 1:

```
mutex.lock()
x++
mutex.unlock()
```

thread 2:

```
mutex.lock()
x++
mutex.unlock()
```

Semaphore

A generalized Mutex that N threads can own at once

A “resource counting” lock

Two methods:

- down (or acquire) ↓
- up (or release) ↑

A Semaphore can be held by N threads, or even by 1 thread N times!

N+1’th down -> block

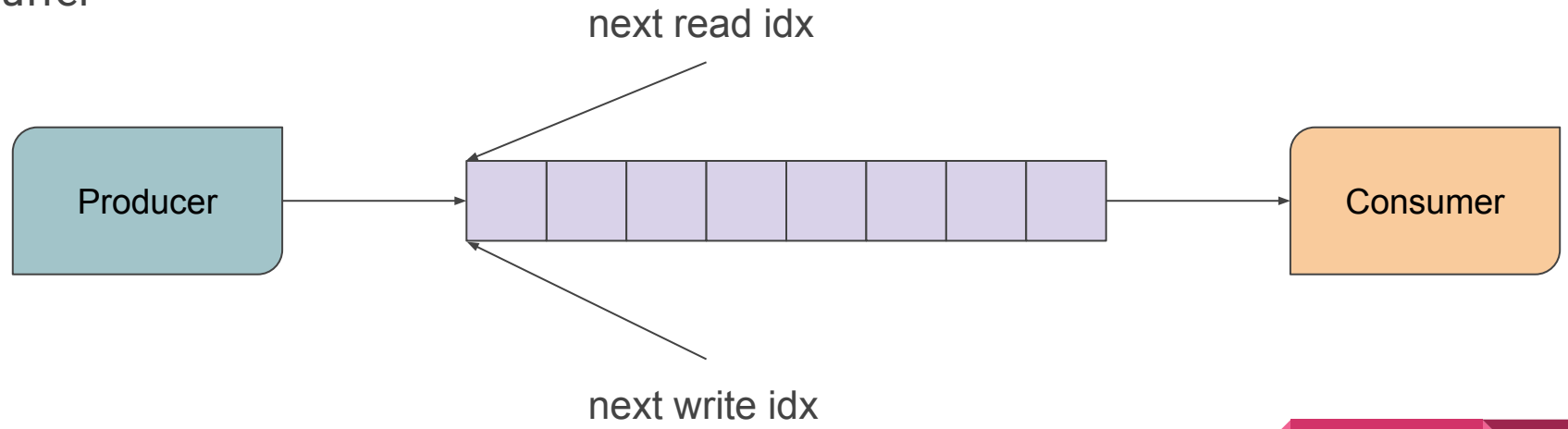
Semaphore

Semaphore(1) == Mutex

Semaphore(2) == a mutex that can be held by 2 threads?

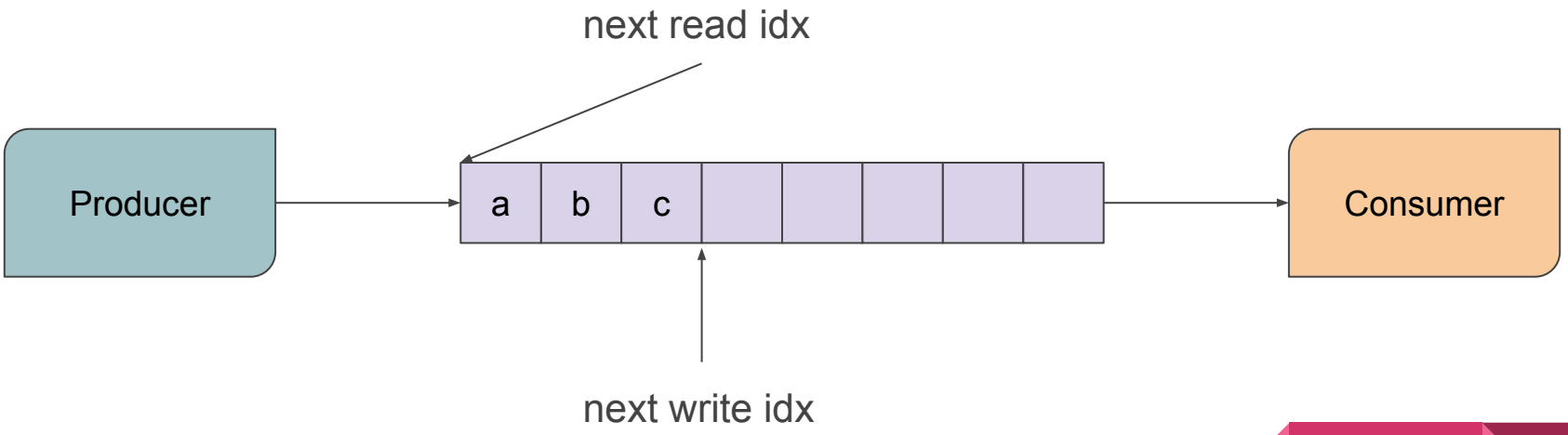
Semaphore example - Bounded Buffer

We have a producer thread and a consumer thread which transfer data with a buffer



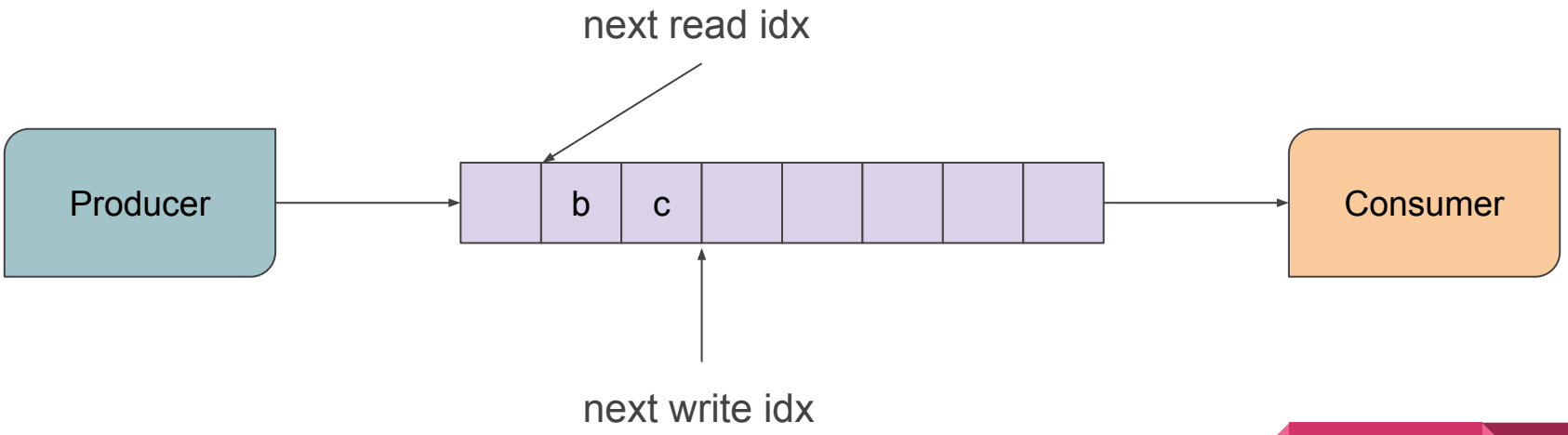
Semaphore example - Bounded Buffer

Producer sends data a, b, c



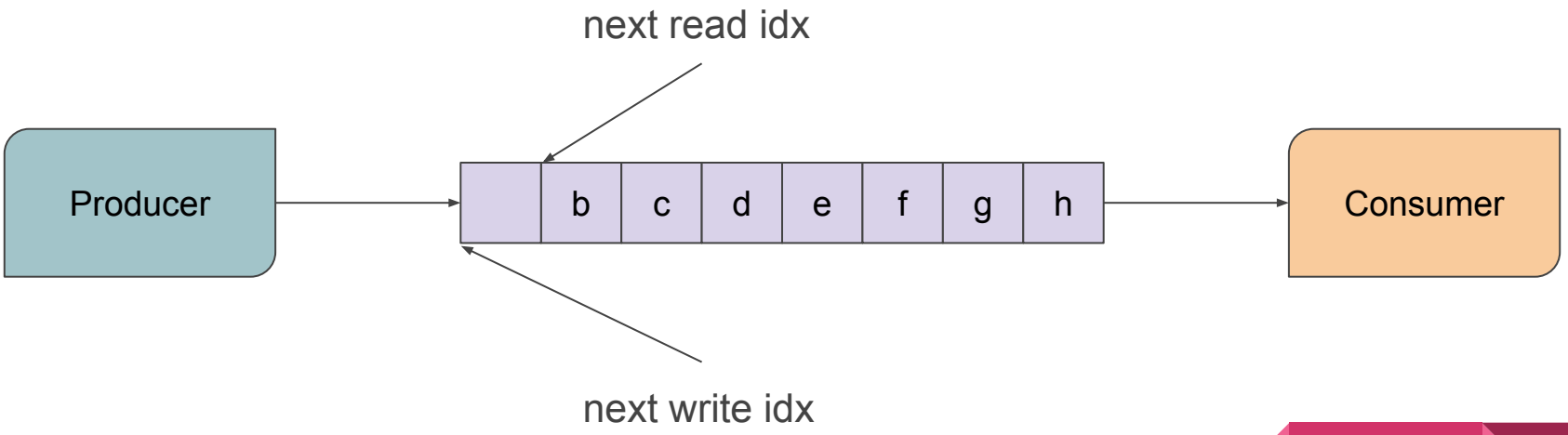
Semaphore example - Bounded Buffer

Consumer reads 1 data (a)



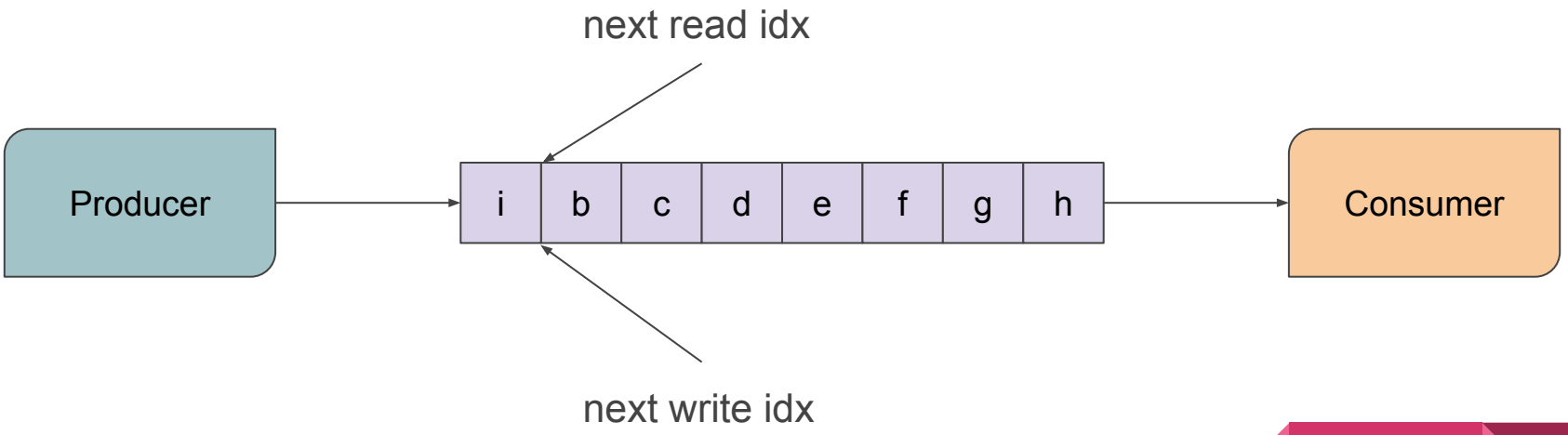
Semaphore example - Bounded Buffer

Producer writes 5 datas (d, e, f, g, h)



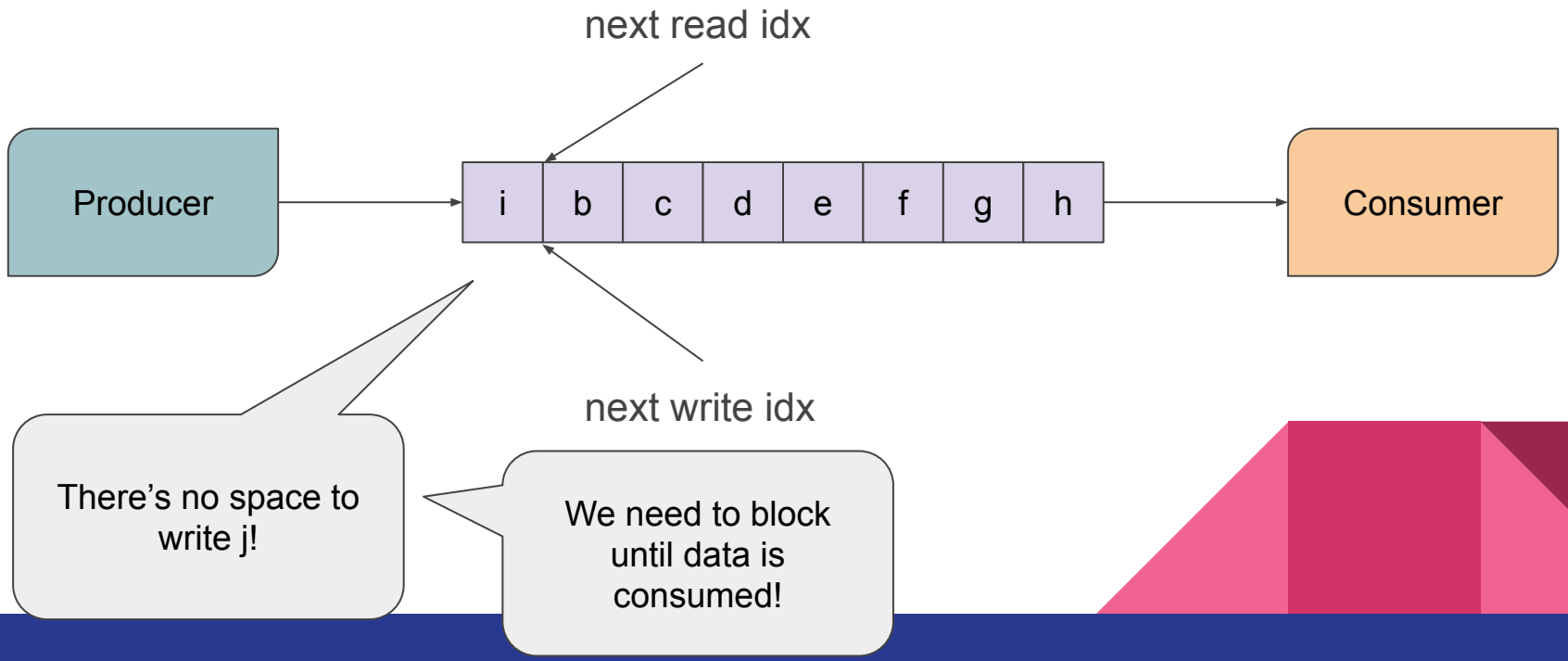
Semaphore example - Bounded Buffer

Producer writes 2 datas (i, j)



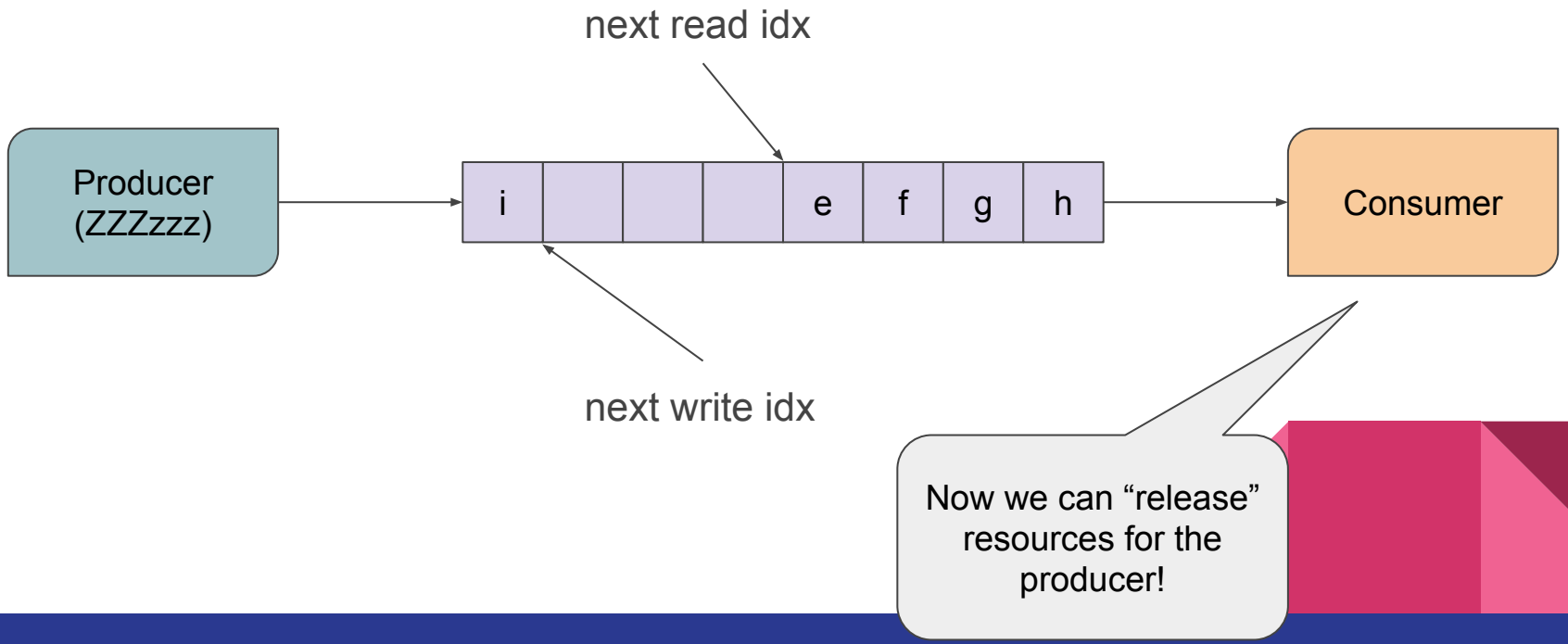
Semaphore example - Bounded Buffer

Producer writes 2 datas (i, j)



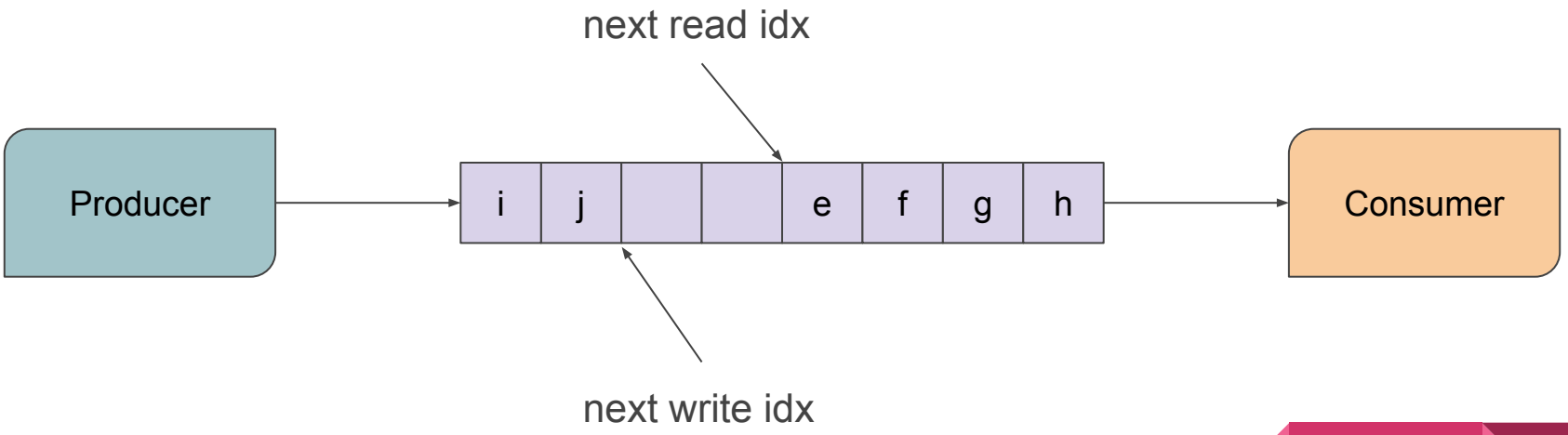
Semaphore example - Bounded Buffer

Consumer reads 3 items (b, c, d)



Semaphore example - Bounded Buffer

Producer can finish writing j



How do we get this sleeping behavior?

A Semaphore(8)!

- Each write into the buffer is a .down()
- Each read from the buffer is a .up()
  - We'd need a separate Semaphore(0) to stop readers from reading an empty buffer!

Barrier

Also known as a “fence”

One method:

- wait

Ensures that N threads all reach the barrier before allowing any to proceed

Barrier example

Implementing array.map(|x| x+2).sum()

- AKA incrementing every element by 2 and then summing the result
- We want to implement this using domain decomposition

Barrier example

thread 1:

```
for idx in 0..N/2:
    array[idx]+=2
s = sum(array)
```

thread 2:

```
for idx in N/2..N:
    array[idx] += 2

… other stuff
```

We need to make sure thread 2 is done looping!

Barrier example

thread 1:

```
for idx in 0..N/2:
    array[idx]+=2
bar.wait()
s = sum(array)
```

thread 2:

```
for idx in N/2..N:
    array[idx] += 2
bar.wait()
… other stuff
```

Barrier Flavors

- One-shot barrier
  - Can only use once per thread
- Reusable barriers
  - Can use as many times as needed (e.g. in a loop)

RWLock

Multiple readers OR one writer lock

Four methods:

- rdlock & rdunlock
- wrlock & wrunlock

Why RWLock?

What if multiple threads just want to read x?

thread 1:

```
mutex.lock()
print(x)
mutex.unlock()
```

thread 2:

```
mutex.lock()
print(x)
mutex.unlock()
```

There is no data race here because there are no writers, so synchronization shouldn't be necessary!

RWLock

Recall the definition of a data race:

- 2 instructions from different threads access the same memory
- at least one is a write
- no synchronization mandating an order between the accesses

RWLock optimizes for this, and allows multiple readers OR one writer

Why RWLock?

thread 1:

```
rwl.rdlock()
print(x)
rwl.rdunlock()
```

thread 2:

```
rwl.rdlock()
print(x)
rwl.rdunlock()
```

Multiple threads can own a rdlock at the same time!

Why RWLock?

thread 1:

```
rwl.rdlock()
print(x)
rwl.rdunlock()
```

thread 3:

```
rwl.wrlock()
x++
rwl.wrunlock()
```

A wrlock cannot be acquired until all rdlocks (and wrlocks) are released!

RWLock

- What if I call rdlock but then accidentally mutate x?
- Gee, wouldn’t it be nice if the compiler enforced that you could never mutate x with only a rdlock held?

Condition Variable

A more complex synchronization primitive that allows blocking until a user-defined condition is met

Three methods:

- wait(Mutex*)
- notify_one
- notify_all

Condition Variable - API

wait(Mutex *m) will atomically release m and block on the CondVar

notify_one() will awaken a thread blocked on the CondVar

- after being woken up, previously waiting threads need to reacquire Mutex

notify_all() will awaken all threads blocked on the CondVar

Condition Variable - Monitor Example

Say we have a variable x. We want to block until x is even. x is protected by Mutex m.

```
def wait_until_x_is_even_broken():
    m.lock()
    if x % 2 == 0:
        print("x is even!")
        m.unlock()
        return
    else:
        cv.wait(m)
        print("x is even!")
        return
```

thread 2:

```
    loop:
        sleep(rand())
        m.lock()
        x++
        cv.notify_one()
        m.unlock()
```

Tell a waiter that x has changed, so it should wake up.

Condition Variable - Monitor Example

Do we know x is even after getting signaled?

```
def wait_until_x_is_even_broken():
    m.lock()
    if x % 2 == 0:
        print("x is even!")
        m.unlock()
        return
    else:
        cv.wait(m)
        print("x is even!")
        return
```

Do we know x is even now?

NO!

Condition Variable - Monitor Example

We must recheck the condition on wakeup!

```
def wait_until_x_is_even_fixed():
    loop:
        m.lock()
        if x % 2 == 0:
            print("x is even!")
            m.unlock()
            return
        else:
            cv.wait(m)
```

Monitor

This design pattern of combining a CondVar and a Mutex is called a Monitor

2 flavors:

- Mesa-style (what we discussed)
- Hoare-style (signal() immediately transfers execution to a waiter, so you don't have to recheck the condition)
  - e.g. for mutex, unlock gives the lock directly to a blocked thread

Every primitive is equivalent

You can implement Mutex using Semaphore

You can implement Semaphore using Mutex

You can implement Barriers using Channels

You can implement an entire Monitor using just Semaphores

Therefore, every primitive has (in theory) the same issues

Deadlock

- One thread calls Mutex.lock() twice without unlocking
- Two threads both call channel.send() expecting the other to receive

Generally catastrophic, one thread is blocked forever

Therefore, every primitive has (in theory) the same issues

Starvation

- Every thread needs a chance to run
- RWLocks with 1000 readers, writers will get starved any runtime

Making the waiter list a FIFO queue can help…

Therefore, every primitive has (in theory) the same issues

Priority Inversion

- High priority thread A is waiting for a lock held by low priority blocked thread B
- B may take a while to run, starving A from runtime that it needs now

Solutions are complex and involve temporarily boosting the priority of B

Therefore, every primitive has (in theory) the same issues

Busy waiting vs blocking

- Busy waiting (aka polling or spinning) wastes energy but is very reactive
- Blocking (context switching) is efficient, but unblocking thread A gives no guarantee on when it actually runs next, -> unpredictable latency

Therefore, every primitive has (in theory) the same issues

Performance

- Synchronizing is generally slow!
  - More accurately, contention on atomic variables is slow

Use the right tool for the job

Even though all synchronization primitives are Turing equivalent, some are much more appropriate than others.

Barrier and Mutex can get you very far. Channels are better when there’s no shared mutable state. CondVars are situationally super helpful.